Recurrent Neural Networks
Mathematical Foundations

Vitor N. Cabral de Oliveira
vnco@cin.ufpe.br

June 9, 2026

This chapter develops the mathematical framework for Recurrent Neural Networks (RNNs), which are designed to process sequential data. We formalise the RNN architecture, derive the forward propagation equations, and present backpropagation through time (BPTT). The problems of vanishing and exploding gradients are analysed, and the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are introduced as effective solutions.

Contents

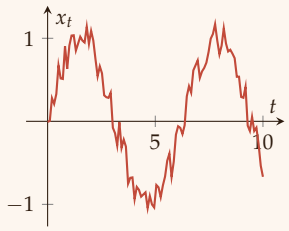
Foundations of Sequence Modeling

Before diving into recurrent architectures, we must establish a rigorous framework for sequence data. Sequences are ordered collections of observations that arise naturally in time series, natural language, audio, video, and many other domains. Understanding their structure and the associated learning problems is essential.

Definition and Notation

Definition 0.1 (Sequence)**.** *A **sequence** of length T is an ordered tuple $\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T)$, where each element $\mathbf{x}_t$ belongs to a set $\mathcal{X}$ (the **observation space**). When $\mathcal{X} \subseteq \mathbb{R}^d$, we speak of a **real-valued vector sequence**.*

- *If T is finite, the sequence is **finite**; if $T = \infty$, it is **infinite**.*
- *When the indices t are discrete (e.g., $t \in \mathbb{N}$), we have a **discrete-time sequence**. Continuous-time sequences are indexed by $t \in \mathbb{R}$ but are less common in machine learning.*



Special cases

- **Univariate sequence**: each $\mathbf{x}_t \in \mathbb{R}$ (scalar). Example: daily temperature.
- **Multivariate sequence**: each $\mathbf{x}_t \in \mathbb{R}^d$, $d > 1$. Example: joint angles of a robotic arm over time.
- **Symbolic sequence**: $\mathcal{X}$ is a finite set of symbols (e.g., characters, words). Example: natural language text.

Probabilistic Description

A sequence can be modelled as a **stochastic process** $\{X_t\}_{t \in \mathcal{T}}$, where each X_t is a random variable taking values in $\mathcal{X}$, and $\mathcal{T}$ is the index set (usually $\mathbb{N}$ or $\{1, \ldots, T\}$). The joint distribution over the entire sequence is $P(X_1 = x_1, \ldots, X_T = x_T)$.

Definition 0.2 (Stationarity)**.** *A stochastic process is **strictly stationary** if for any $t_1, \ldots, t_k$ and any shift τ,*

$$P(X_{t_1} \leq x_1, \ldots, X_{t_k} \leq x_k) = P(X_{t_1+\tau} \leq x_1, \ldots, X_{t_k+\tau} \leq x_k).$$

*It is **weakly stationary** (or covariance stationary) if:*

1. $\mathbb{E}[X_t] = \mu$ *(constant mean),*
2. $\mathrm{Var}(X_t) = \sigma^2$ *(constant variance),*

3. $\mathrm{Cov}(X_t, X_{t+\tau})$ *depends only on* τ *(the lag).*

Stationarity simplifies modeling because the statistical properties do not change over time. Many real-world sequences are non-stationary (e.g., financial returns have time-varying volatility).

Auto-correlation and Memory

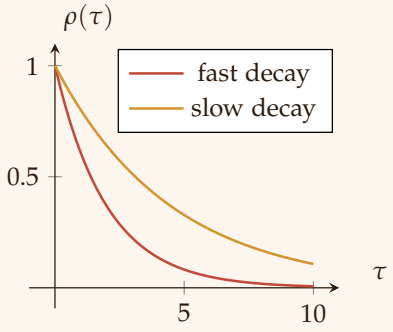
A key characteristic of sequences is that observations are not independent; they exhibit **temporal dependence**.

Definition 0.3 (Auto-correlation Function (ACF))**.** *For a weakly stationary sequence with variance* σ^2*, the auto-correlation at lag* τ *is*

$$\rho(\tau) = \frac{\mathrm{Cov}(X_t, X_{t+\tau})}{\sigma^2}.$$

The ACF satisfies $\rho(0) = 1$*,* $|\rho(\tau)| \leq 1$*, and* $\rho(\tau) = \rho(-\tau)$*.*

If $\rho(\tau)$ decays slowly (e.g., $\rho(\tau) \sim e^{-\alpha\tau}$ or power-law), the sequence has **long-term memory**. Short-memory processes have $\rho(\tau) \to 0$ exponentially fast.



Markov Property and Order

A fundamental simplification for sequence modeling is the Markov assumption: the future depends only on a finite past.

Definition 0.4 (Markov Chain)**.** *A discrete-time stochastic process* $\{X_t\}$ *is a* ***Markov chain of order*** *k if for all t,*

$$P(X_t = x_t \mid X_{1:t-1} = x_{1:t-1}) = P(X_t = x_t \mid X_{t-k:t-1} = x_{t-k:t-1}),$$

i.e., the conditional distribution of the next state depends only on the k most recent states. For $k = 1$*, it is a* ***first-order Markov chain****.*

Theorem 0.1 (Markov Chain Decomposition)**.** *For a first-order Markov chain, the joint distribution factorizes as*

$$P(X_{1:T} = x_{1:T}) = P(X_1 = x_1) \prod_{t=2}^{T} P(X_t = x_t \mid X_{t-1} = x_{t-1}).$$

The Markov property is the theoretical backbone of many sequence models, including RNNs (which approximate an infinite-order Markov chain via hidden states).

Deep Dive: Markov Models

Discrete-Time Markov Chains A discrete-time stochastic process $\{X_t\}_{t=1}^{\infty}$ with state space $\mathcal{S}$ (finite or countable) is a **Markov chain** if for all $t \geq 2$ and all $s_1, \ldots, s_t \in \mathcal{S}$,

$$P(X_t = s_t \mid X_{t-1} = s_{t-1}, \ldots, X_1 = s_1) = P(X_t = s_t \mid X_{t-1} = s_{t-1}).$$

The chain is **time-homogeneous** if the transition probability does not depend on t:

$$P(X_t = j \mid X_{t-1} = i) = T_{ij}, \quad \forall t,$$

where $\mathbf{T}$ is the **transition matrix** with rows summing to 1.

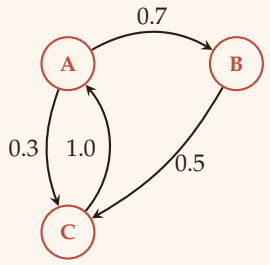


Figure 3: A three-state Markov chain with transition probabilities.

Definition 0.5 (Order k Markov Chain)**.** *A process satisfies the **Markov property of order** k if*

$$P(X_t \mid X_{1:t-1}) = P(X_t \mid X_{t-k:t-1}).$$

The state can be augmented to $\tilde{X}_t = (X_{t-k+1}, \ldots, X_t)$, which forms a first-order chain on a larger state space of size $|\mathcal{S}|^k$.

Key Properties of Markov Chains

- **Stationary distribution**: If a distribution $\boldsymbol{\pi}$ satisfies $\boldsymbol{\pi}^\top = \boldsymbol{\pi}^\top \mathbf{T}$, it is stationary; for an ergodic chain, $\lim_{n\to\infty} \mathbf{T}^n$ has rows equal to $\boldsymbol{\pi}$.
- **Ergodicity**: A chain that is aperiodic and positive recurrent converges to a unique stationary distribution regardless of initial state.
- **Likelihood of a sequence**: $P(X_{1:T} = s_{1:T}) = P(X_1 = s_1) \prod_{t=2}^{T} T_{s_{t-1}, s_t}$.

Hidden Markov Models (HMMs) In many real-world sequences, the underlying state is not directly observable. HMMs extend Markov chains by assuming that each state X_t emits an observation O_t through a known emission distribution.

Definition 0.6 (Hidden Markov Model)**.** *An HMM is a doubly stochastic process $(X_t, O_t)_{t=1}^{T}$ where:*

1. *$\{X_t\}$ is a time-homogeneous Markov chain with state space $\mathcal{S}$, initial distribution $\boldsymbol{\pi}$, and transition matrix $\mathbf{T}$.*
2. *Given X_t, the observation O_t is independent of all other states and observations, with emission probability $E(o \mid X_t = s) = P(O_t = o \mid X_t = s)$.*

The joint distribution of a sequence of states and observations is

$$P(X_{1:T} = s_{1:T}, O_{1:T} = o_{1:T}) = \pi_{s_1} \prod_{t=2}^{T} T_{s_{t-1}, s_t} \prod_{t=1}^{T} E(o_t \mid s_t).$$

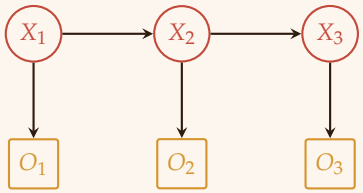


Figure 4: Graphical model of an HMM: hidden states X_t (circle) emit observations O_t (square).

Inference and Learning in HMMs Three fundamental problems for HMMs are solved with efficient dynamic programming algorithms.

1. Likelihood Computation (Forward Algorithm) Given observations $o_{1:T}$, compute $P(O_{1:T} = o_{1:T}) = \sum_{s_{1:T}} P(X_{1:T}, O_{1:T})$. The forward recursion:

$$\alpha_1(s) = \pi_s \, E(o_1 \mid s), \quad \alpha_t(s) = E(o_t \mid s) \sum_{s'} \alpha_{t-1}(s') \, T_{s', s}.$$

Then $P(O_{1:T}) = \sum_s \alpha_T(s)$.

2. Most Likely State Sequence (Viterbi) Find $\arg\max_{s_{1:T}} P(X_{1:T} \mid O_{1:T})$. The Viterbi algorithm uses:

$$\delta_1(s) = \pi_s E(o_1 \mid s), \quad \delta_t(s) = E(o_t \mid s) \max_{s'} \delta_{t-1}(s') T_{s',s},$$

with backpointers to reconstruct the optimal path.

3. Parameter Estimation (Baum-Welch / EM) Given observations but no states, we maximise $\log P(O_{1:T} \mid \boldsymbol{\theta})$ via the EM algorithm. The E-step computes posterior probabilities $\gamma_t(s) = P(X_t = s \mid O_{1:T})$ and $\xi_t(s', s) = P(X_{t-1} = s', X_t = s \mid O_{1:T})$ using the forward-backward algorithm. The M-step updates:

$$\pi_s \propto \gamma_1(s), \quad T_{s',s} \propto \frac{\sum_{t=2}^{T} \xi_t(s', s)}{\sum_{t=2}^{T} \gamma_{t-1}(s')}, \quad E(o \mid s) \propto \frac{\sum_{t:O_t=o} \gamma_t(s)}{\sum_t \gamma_t(s)}.$$

Theorem 0.2 (Baum-Welch Convergence)**.** *The Baum-Welch algorithm monotonically increases the likelihood $P(O_{1:T} \mid \boldsymbol{\theta})$ and converges to a local maximum of the likelihood.*

Markov Decision Processes (MDPs) - A Brief Mention When actions influence state transitions, we obtain an MDP. While not purely generative, MDPs are foundational for reinforcement learning. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$ where:

- $\mathcal{S}$: state space,
- $\mathcal{A}$: action space,
- $\mathcal{T}(s' \mid s, a)$: transition probability,
- $\mathcal{R}(s, a)$: immediate reward,
- $\gamma \in [0, 1]$: discount factor.

The goal is to find a policy $\pi(a \mid s)$ that maximises expected cumulative reward.

From Hidden Markov Models to Recurrent Neural Networks HMMs and RNNs both address sequence modeling but differ fundamentally:

- **State representation**: HMM states are discrete (finite $\mathcal{S}$); RNN states are continuous vectors $\mathbf{h}_t \in \mathbb{R}^{d_h}$. Continuous states allow exponentially more expressive power for a fixed dimensionality.
- **Transition function**: HMM transitions are linear (matrix multiplication on a simplex); RNN transitions are nonlinear (e.g., $\tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$), enabling the modelling of complex dynamics.

- **Emission model**: HMM emissions factorize as $E(o_t \mid s_t)$; RNNs can produce outputs that depend on the entire history via $\mathbf{h}_t$.
- **Learning**: HMMs are trained with EM (Baum-Welch), which guarantees local likelihood increase; RNNs use BPTT and stochastic gradient descent, which scale better to large state spaces and long sequences.
- **Long-range dependencies**: HMMs suffer from exponentially many states to represent k-th order dependencies; RNNs attempt to compress history into a fixed-size vector (though suffer from vanishing gradients).

Theorem 0.3 (HMM as a Linear RNN)**.** *If the state vector $\mathbf{h}_t$ is taken as the vector of probabilities $P(X_t = s \mid \text{history})$, then the update $\mathbf{h}_t \propto \mathbf{E}_{o_t} \odot (\mathbf{T}^\top \mathbf{h}_{t-1})$ (normalised) is a linear recurrence (with a non-linear normalisation). This is equivalent to the forward algorithm of an HMM.*

The inadequacies of HMMs for large-scale sequence modeling - discrete state, linear transitions, independence of observations given state - motivated the development of more powerful models, including conditional random fields (CRFs) and ultimately recurrent neural networks. However, HMMs remain important for their interpretability, small-sample efficiency, and theoretical guarantees.

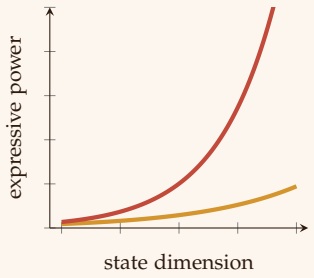


Figure 5: Continuous RNN states (red line) can represent exponentially more distinct histories than discrete HMM states (orange line) of the same dimension.

Sequence Learning Tasks

We categorise learning problems based on inputs and outputs:

1. **Sequence prediction (auto-regressive)**: Given past observations $x_{1:t-1}$, predict x_t. The model learns $P(x_t \mid x_{1:t-1})$.
2. **Sequence classification**: Given the entire input sequence $\mathbf{x}_{1:T}$, predict a single label y. Example: sentiment analysis.
3. **Sequence labelling (per-time-step classification)**: Given $\mathbf{x}_{1:T}$, predict a label y_t for each t. Example: part-of-speech tagging.
4. **Sequence generation**: Sample a new sequence from the learned distribution $P(x_{1:T})$.
5. **Sequence-to-sequence (seq2seq)**: Map an input sequence $\mathbf{x}_{1:T}$ to an output sequence $\mathbf{y}_{1:T'}$ (possibly of different length). Example: machine translation.

Why Feedforward Networks Fail for Sequences

Feedforward (multilayer perceptron) networks assume fixed-length inputs and outputs, and - critically - they treat each input as independent. To process a sequence of length T, one could:

1. **Concatenate all elements** into a single $d \times T$ vector. This fails because T is variable and the resulting dimension becomes enormous.
2. **Slide a window** of fixed size w over the sequence, using the last w values as input to a feedforward network. This ignores dependencies longer than w (finite memory) and does not share parameters across time positions efficiently.

Moreover, feedforward networks have no built-in mechanism to handle the sequential order or to maintain a state that evolves over time. They cannot capture the inductive bias that the same transformation applies at each time step (weight sharing) - a property that recurrent networks exploit.

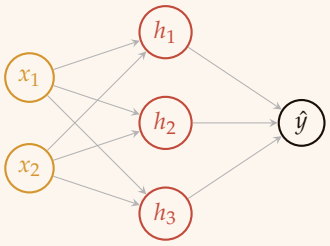


Figure 6: Feedforward networks treat each time step independently; they cannot share weights across time or maintain a dynamic state.

Recurrence as a Natural Solution

An RNN addresses these limitations by introducing a **hidden state** $\mathbf{h}_t$ that evolves over time according to

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}),$$

where f is a parameterised function (same at each step). This gives three crucial properties:

- **Variable-length handling**: the same recurrence applies for any T.
- **Parameter sharing**: the same weights $\boldsymbol{\theta}$ are used at each time step, reducing the number of parameters and promoting generalization.
- **Memory**: the hidden state $\mathbf{h}_t$ can theoretically retain information from arbitrarily distant past (though in practice limited by vanishing gradients).

Thus, the RNN is a natural and principled extension of feedforward networks to sequential data. The remainder of this chapter formalises the RNN architecture, training, and its advanced variants.

The Elman RNN Architecture

The most basic RNN, proposed by Elman (1990), consists of an input layer, a hidden layer with recurrent connections, and an output layer. At each time step t, the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ is updated based on the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$.

Definition 0.7 (RNN Cell). *A simple RNN cell is defined by the following equations:*

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h, \tag{1}$$
$$\mathbf{h}_t = \phi_h(\mathbf{a}_t), \tag{2}$$
$$\mathbf{o}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \tag{3}$$
$$\hat{\mathbf{y}}_t = \phi_y(\mathbf{o}_t), \tag{4}$$

where:

- $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$ *is the input-to-hidden weight matrix,*
- $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$ *is the recurrent (hidden-to-hidden) weight matrix,*
- $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ *is the hidden-to-output weight matrix,*
- $\mathbf{b}_h \in \mathbb{R}^{d_h}$, $\mathbf{b}_y \in \mathbb{R}^{d_y}$ *are bias vectors,*
- ϕ_h *is a nonlinear activation function (typically* tanh *or ReLU),*
- ϕ_y *is the output activation (e.g., softmax for classification, identity for regression).*

The initial hidden state $\mathbf{h}_0$ *is usually set to the zero vector, but can also be learned.*

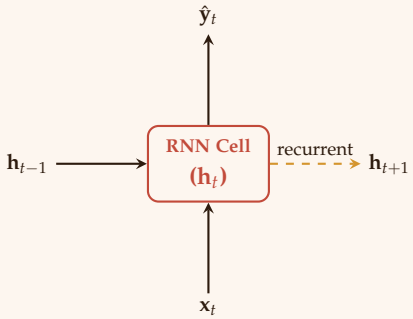


Figure 7: Schematic of a single RNN cell. The hidden state $\mathbf{h}_t$ propagates to the next time step.

Connection to Linear Dynamical Systems

Classical state-space models, known as linear dynamical systems (LDS) or Kalman filters, provide a linear counterpart to RNNs.

Definition 0.8 (Linear Dynamical System). *A discrete-time LDS is defined by:*

$$\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}\mathbf{x}_t + \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{Q}), \tag{5}$$
$$\mathbf{y}_t = \mathbf{C}\mathbf{h}_t + \delta_t, \quad \delta_t \sim \mathcal{N}(0, \mathbf{R}), \tag{6}$$

where $\mathbf{A}, \mathbf{B}, \mathbf{C}$ *are matrices, and* $\mathbf{h}_t$ *is the latent state.*

Comparing with the Elman RNN:

$$\mathbf{h}_t = \phi_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h).$$

The RNN generalises the LDS in two ways:

- Nonlinear activation ϕ_h (instead of identity),
- Deterministic state (no process noise), though one can add noise for probabilistic RNNs.

If ϕ_h is the identity function and $\mathbf{b}_h = \mathbf{0}$, the RNN reduces to a linear system. Conversely, the LDS can be interpreted as a linear RNN with Gaussian noise. Inference in LDS (Kalman smoothing) resembles forward-backward propagation in RNNs but with exact uncertainty propagation. This connection is valuable for understanding RNNs as flexible nonlinear extensions of classical time-series models.

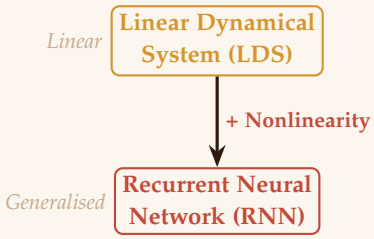


Figure 8: RNNs generalise linear dynamical systems by replacing linear transitions with nonlinear functions.

Bidirectional Recurrent Neural Networks

Many sequence tasks - such as part-of-speech tagging or named entity recognition - benefit from access to both past and future context. A bidirectional RNN (BiRNN) processes the sequence in two directions.

Definition 0.9 (Bidirectional RNN)**.** *Given an input sequence* $\mathbf{x}_{1:T}$*, a BiRNN maintains:*

- *A forward hidden state* $\overrightarrow{\mathbf{h}}_t = RNN_{fwd}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1})$,
- *A backward hidden state* $\overleftarrow{\mathbf{h}}_t = RNN_{bwd}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1})$,
- *A combined state* $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ *(concatenation) or* $\mathbf{h}_t = \overrightarrow{\mathbf{h}}_t + \overleftarrow{\mathbf{h}}_t$.

The output at time t *is computed from* $\mathbf{h}_t$*:* $\hat{\mathbf{y}}_t = \phi_y(\mathbf{W}_y\mathbf{h}_t + \mathbf{b}_y)$.

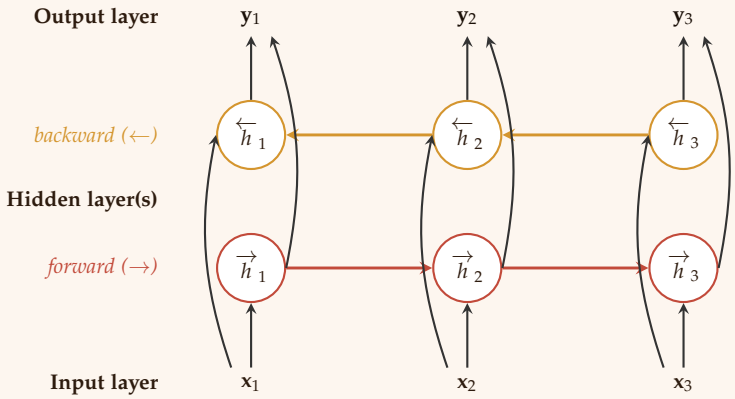


Figure 9: Structure of a Bidirectional Recurrent Neural Network (BiRNN). Inputs $\mathbf{x}_t$ independently drive the forward and backward hidden states, which are then combined cleanly to project the outputs $\mathbf{y}_t$.

During training, we unroll both directions and compute the loss at each step. Backpropagation through time propagates gradients separately through the forward and backward chains, and the concatenated state gradients are summed. Bidirectional RNNs are not suitable for real-time or online tasks (since the full sequence is required to compute backward states), but they are very effective for offline sequence labelling.

Deep (Stacked) Recurrent Neural Networks

To increase representational capacity, multiple RNN layers can be stacked vertically. At each time step t, the output of layer $\ell - 1$ becomes the input to layer ℓ.

Definition 0.10 (Stacked RNN)**.** *Let $\mathbf{h}_t^{(\ell)}$ denote the hidden state of layer ℓ at time t, with $\mathbf{h}_t^{(0)} = \mathbf{x}_t$. For $\ell = 1, \dots, L$,*

$$\mathbf{a}_t^{(\ell)} = \mathbf{W}_{xh}^{(\ell)} \mathbf{h}_t^{(\ell-1)} + \mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}_h^{(\ell)}, \quad \mathbf{h}_t^{(\ell)} = \phi_h(\mathbf{a}_t^{(\ell)}).$$

The output is then $\hat{\mathbf{y}}_t = \phi_y(\mathbf{W}_{hy} \mathbf{h}_t^{(L)} + \mathbf{b}_y)$.

Training deep RNNs requires backpropagation through both time and layers. The gradient flows downwards through the layer hierarchy and backwards in time. Vanishing gradients are exacerbated with depth, so gated architectures (LSTM, GRU) are typically used in deep stacks. Residual connections (skip connections across layers) can also help.

Algorithm 1: Forward pass for a stacked RNN (2 layers)

$\mathbf{h}_0^{(1)} \leftarrow \mathbf{0}$, $\mathbf{h}_0^{(2)} \leftarrow \mathbf{0}$
for $t = 1$ to T **do**
  $\mathbf{a}_t^{(1)} = \mathbf{W}_{xh}^{(1)} \mathbf{x}_t + \mathbf{W}_{hh}^{(1)} \mathbf{h}_{t-1}^{(1)} + \mathbf{b}_h^{(1)}$
  $\mathbf{h}_t^{(1)} = \phi_h(\mathbf{a}_t^{(1)})$
  $\mathbf{a}_t^{(2)} = \mathbf{W}_{xh}^{(2)} \mathbf{h}_t^{(1)} + \mathbf{W}_{hh}^{(2)} \mathbf{h}_{t-1}^{(2)} + \mathbf{b}_h^{(2)}$
  $\mathbf{h}_t^{(2)} = \phi_h(\mathbf{a}_t^{(2)})$
  $\hat{\mathbf{y}}_t = \phi_y(\mathbf{W}_{hy} \mathbf{h}_t^{(2)} + \mathbf{b}_y)$
end for

Deep RNNs are used in applications such as acoustic modelling for speech recognition and machine translation, where hierarchical feature extraction is beneficial.

Unrolling Through Time

To understand training, it is convenient to "unroll" the recurrence over time, obtaining a deep feedforward network with shared weights across layers (each layer corresponds to a time step). The unrolled architecture for T time steps is shown in Figure 10.

Forward Propagation

Given an input sequence $\mathbf{x}_{1:T}$, forward propagation computes the hidden states and outputs sequentially:

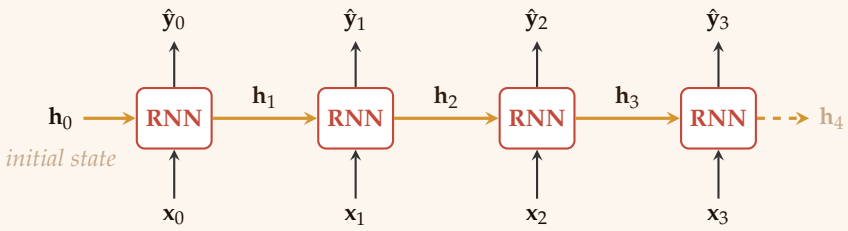


Figure 10: Unrolled RNN for $T = 4$ time steps. The same weights $\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}$ are reused at every step, with hidden states $\mathbf{h}_t$ propagating horizontally.

Algorithm 2: Forward pass of an RNN

$\mathbf{h}_0 \leftarrow \mathbf{0}$ (or learned initial state)
for $t = 1$ to T **do**
  $\mathbf{a}_t \leftarrow \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h$
  $\mathbf{h}_t \leftarrow \phi_h(\mathbf{a}_t)$
  $\mathbf{o}_t \leftarrow \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$
  $\hat{\mathbf{y}}_t \leftarrow \phi_y(\mathbf{o}_t)$
end for

Training RNNs: Backpropagation Through Time (BPTT)

RNNs are trained by minimising a loss function that aggregates errors over all time steps. For a sequence with targets $\mathbf{y}_{1:T}$, the total loss is the sum of per-time-step losses:

$$\mathcal{L}(\mathbf{y}_{1:T}, \hat{\mathbf{y}}_{1:T}) = \sum_{t=1}^{T} \ell(\mathbf{y}_t, \hat{\mathbf{y}}_t),$$

where ℓ is a pointwise loss (e.g., cross-entropy or squared error). Because the same parameters are shared across time, we must back-propagate gradients through the entire unrolled network. This procedure is called **backpropagation through time (BPTT)**.

Gradient Computation

Let θ denote any trainable parameter (e.g., $w_{ij}^{(xh)}$). The total derivative is

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial \ell_t}{\partial \theta},$$

where ℓ_t is the loss at time t. For parameters that appear at every time step (like $\mathbf{W}_{hh}$), the contribution from each t depends on all future time steps because the hidden state at time t influences all subsequent losses.

We derive the gradients for $\mathbf{W}_{hh}$; similar reasoning applies to $\mathbf{W}_{xh}$, $\mathbf{b}_h$, $\mathbf{W}_{hy}$, $\mathbf{b}_y$.

Error Signals Define the error at the output of the RNN at time t as:

$$\delta_t^{\text{out}} = \frac{\partial \ell_t}{\partial \mathbf{o}_t} = \nabla_{\hat{\mathbf{y}}_t} \ell_t \odot \phi_y'(\mathbf{o}_t).$$

For the hidden layer, we need the sensitivity of the total future loss with respect to the pre-activation $\mathbf{a}_t$. By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \underbrace{\frac{\partial \ell_t}{\partial \mathbf{a}_t}}_{\text{immediate}} + \underbrace{\frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_t}}_{\text{from future}}.$$

The immediate contribution is $\frac{\partial \ell_t}{\partial \mathbf{a}_t} = \mathbf{W}_{hy}^\top \delta_t^{\text{out}} \odot \phi_h'(\mathbf{a}_t)$.

The future contribution comes from the fact that $\mathbf{h}_t$ affects $\mathbf{a}_{t+1}$ via $\mathbf{W}_{hh}\mathbf{h}_t$. Hence,

$$\frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_t} = \left(\frac{\partial \mathbf{a}_{t+1}}{\partial \mathbf{h}_t}\right)^\top \frac{\partial \mathcal{L}_{t+1:T}}{\partial \mathbf{a}_{t+1}} = \mathbf{W}_{hh}^\top \delta_{t+1} \odot \phi_h'(\mathbf{a}_t),$$

where we define $\delta_t = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t}$. Thus, the backward recurrence is:

$$\delta_T = \mathbf{W}_{hy}^\top \delta_T^{\text{out}} \odot \phi_h'(\mathbf{a}_T), \tag{7}$$

$$\delta_t = \left(\mathbf{W}_{hy}^\top \delta_t^{\text{out}} + \mathbf{W}_{hh}^\top \delta_{t+1}\right) \odot \phi_h'(\mathbf{a}_t), \quad t = T-1, \ldots, 1. \tag{8}$$

Gradients with Respect to Weights Once the δ_t are known, the gradients are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \delta_t \, \mathbf{h}_{t-1}^\top, \tag{9}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \delta_t \, \mathbf{x}_t^\top, \tag{10}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_h} = \sum_{t=1}^{T} \delta_t, \tag{11}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \delta_t^{\text{out}} \, \mathbf{h}_t^\top, \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_y} = \sum_{t=1}^{T} \delta_t^{\text{out}}. \tag{13}$$

These gradients are then used in any gradient-based optimiser (SGD, Adam, etc.). The complete BPTT algorithm is summarised below.

Vanishing and Exploding Gradients in Depth

We now analyse the gradient propagation problem more rigorously. Recall the hidden state recurrence $\mathbf{h}_t = \phi_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$.

Algorithm 3: Backpropagation Through Time (BPTT)

1: Run forward pass to store $\mathbf{a}_t, \mathbf{h}_t, \mathbf{o}_t$ for $t = 1, \dots, T$.
2: Compute $\delta_T^{\text{out}} = \nabla_{\hat{\mathbf{y}}_T} \ell_T \odot \phi_y'(\mathbf{o}_T)$.
3: $\delta_T = \mathbf{W}_{hy}^\top \delta_T^{\text{out}} \odot \phi_h'(\mathbf{a}_T)$.
4: **for** $t = T-1$ down to 1 **do**
5: $\quad \delta_t^{\text{out}} = \nabla_{\hat{\mathbf{y}}_t} \ell_t \odot \phi_y'(\mathbf{o}_t)$
6: $\quad \delta_t = \left(\mathbf{W}_{hy}^\top \delta_t^{\text{out}} + \mathbf{W}_{hh}^\top \delta_{t+1}\right) \odot \phi_h'(\mathbf{a}_t)$
7: **end for**
8: Accumulate gradients for $\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h, \mathbf{W}_{hy}, \mathbf{b}_y$ using the formulas above.
9: Update parameters with gradient descent.

For notational simplicity, define the pre-activation $\mathbf{a}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h$, so $\mathbf{h}_t = \phi_h(\mathbf{a}_t)$. The Jacobian of $\mathbf{h}_t$ with respect to $\mathbf{h}_{t-1}$ is

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^\top \operatorname{diag}(\phi_h'(\mathbf{a}_t)).$$

Let $\mathbf{J}_t = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$. For a loss $\mathcal{L}$ that depends on the whole sequence, the gradient of $\mathcal{L}$ with respect to $\mathbf{h}_{t-1}$ is obtained by backpropagating through time:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} = \sum_{k=t}^{T} \left(\prod_{i=t}^{k} \mathbf{J}_i\right)^\top \frac{\partial \mathcal{L}}{\partial \mathbf{h}_k}.$$

For a term k steps ahead, the product involves $k - t + 1$ Jacobians. Taking norms and using submultiplicativity,

$$\left\| \prod_{i=t}^{k} \mathbf{J}_i \right\| \leq \prod_{i=t}^{k} \|\mathbf{W}_{hh}^\top\| \cdot \|\operatorname{diag}(\phi_h'(\mathbf{a}_i))\|.$$

If $\phi_h = \tanh$, then $|\tanh'(z)| \leq 1$, so $\|\operatorname{diag}(\phi_h'(\mathbf{a}_i))\| \leq 1$. Hence,

$$\left\| \prod_{i=t}^{k} \mathbf{J}_i \right\| \leq \|\mathbf{W}_{hh}\|^{k-t+1}.$$

Theorem 0.4 (Exponential Gradient Decay)**.** *Let $\rho(\mathbf{W}_{hh})$ be the spectral radius of $\mathbf{W}_{hh}$. For a simple RNN with $\phi_h = \tanh$,*

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} \right\| \leq \sum_{k=t}^{T} \rho(\mathbf{W}_{hh})^{k-t+1} \left\| \frac{\partial \mathcal{L}}{\partial \mathbf{h}_k} \right\|.$$

If $\rho(\mathbf{W}_{hh}) < 1$, gradients vanish exponentially with the distance $k - t + 1$. If $\rho(\mathbf{W}_{hh}) > 1$, gradients can explode.

The vanishing gradient problem is the primary reason simple RNNs cannot learn long-range dependencies (e.g., more than 10-20 steps). Gated architectures (LSTM, GRU) mitigate this by allowing the gradient to bypass the multiplicative chain via additive updates.

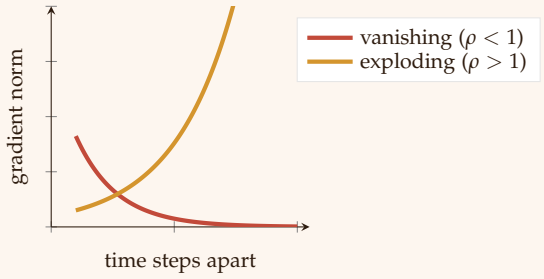


Figure 11: Exponential behaviour of gradient magnitude with sequence distance: vanishing and exploding.

Gradient Clipping When gradients explode, the parameter update becomes erratic and may cause overflow. A simple but effective remedy is **gradient clipping**. Two common variants exist:

Definition 0.11 (Norm-based Gradient Clipping)**.** *Let $\mathbf{g} = \nabla_\theta \mathcal{L}$ be the gradient vector of all parameters. If $\|\mathbf{g}\| > c$, rescale it:*

$$\tilde{\mathbf{g}} = \frac{c}{\|\mathbf{g}\|}\mathbf{g}.$$

Here $c > 0$ is a user-defined threshold. This preserves the direction while bounding the step size.

Definition 0.12 (Value-based Gradient Clipping)**.** *For each component g_i, set $\tilde{g}_i = \max(-c, \min(c, g_i))$. This is elementwise clipping.*

Norm clipping is preferred in RNNs because it respects the geometry of the parameter space. The following result guarantees stability:

Theorem 0.5 (Convergence with Clipping)**.** *Under standard assumptions (bounded gradients, Lipschitz loss), gradient clipping ensures that the parameter iterates remain bounded and that stochastic gradient descent converges almost surely to a stationary point, even when the unclipped gradients would diverge.*

In practice, c is often set between 1 and 5 for RNNs. Clipping is applied before the parameter update.

Gated Recurrent Architectures

To mitigate the vanishing gradient problem, more sophisticated RNN cells introduce **gates** that control the flow of information.

Long Short-Term Memory (LSTM)

The LSTM, proposed by Hochreiter and Schmidhuber (1997), maintains a separate cell state $\mathbf{c}_t$ in addition to the hidden state $\mathbf{h}_t$. The cell state acts as a "conveyor belt" that can carry information over many steps. Three gates control access:

- **Forget gate**: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
- **Input gate**: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Output gate**: $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$

A candidate cell update is $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$. Then the cell state and hidden state are updated:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{14}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{15}$$

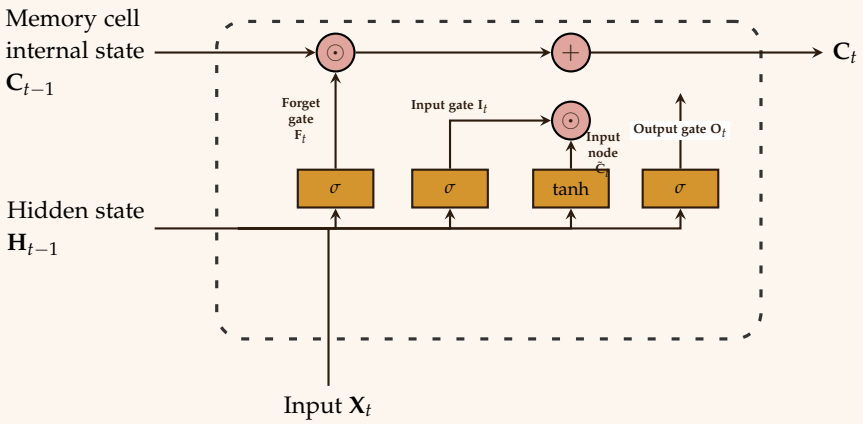


Figure 12: Corrected schematic of an LSTM cell. The memory flow arrows connect directly to the text variables, and all gate labels are mathematically centered along their internal routing lines.

The forget gate allows the network to selectively erase the past; the input gate decides which new information to store; the output gate controls what is exposed as the hidden state.

Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) simplifies the LSTM by combining the forget and input gates into a single **update gate** and merging the cell and hidden states. Its equations are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \quad \text{(update gate)}, \tag{16}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \quad \text{(reset gate)}, \tag{17}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \tag{18}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{19}$$

GRUs have fewer parameters than LSTMs and often perform similarly.

Connection to Maximum Likelihood Estimation

For sequence prediction (e.g., language modeling), the RNN defines a conditional probability distribution over the next token given the history:

$$P(\mathbf{y}_t \mid \mathbf{x}_{1:t}, \mathbf{y}_{1:t-1}) = \text{softmax}(\mathbf{o}_t).$$

The total likelihood of a sequence is $\prod_{t=1}^{T} P(\mathbf{y}_t \mid \text{history})$. Minimising the cross-entropy loss is therefore equivalent to maximum likelihood estimation under the model.

Practical Considerations and Extensions

- **Truncated BPTT**: For very long sequences, full BPTT is expensive. Truncated BPTT splits the sequence into blocks and only back-propagates a fixed number of steps (k) inside each block. This approximates the true gradient and is widely used.
- **Teacher forcing**: During training, the true previous target $\mathbf{y}_{t-1}$ (or a sample) is fed as input instead of the model's own prediction. This speeds up training but may cause exposure bias.
- **Deep RNNs**: Stacking multiple RNN layers (each layer's hidden state becomes input to the next layer) increases representational capacity.
- **Bidirectional RNNs**: Two RNNs process the sequence forward and backward, and their hidden states are concatenated. This provides context from both past and future, useful for sequence tagging.

Algorithm 4: Truncated BPTT (with block size k)

1: Initialise $\mathbf{h}_0$.
2: **for** each block b of k consecutive time steps **do**
3: Run forward pass for k steps, storing intermediate values.
4: Compute loss for the block.
5: Backpropagate gradients only through the k steps (do not propagate beyond the block start).
6: Update parameters.
7: Carry over $\mathbf{h}_{b \cdot k}$ as initial state for the next block.
8: **end for**

Attention Mechanisms

A limitation of RNNs (even with gating) is that the fixed-dimensional hidden state $\mathbf{h}_t$ must compress the entire history. For long sequences, this bottleneck causes information loss. Attention mechanisms overcome this by allowing the decoder to directly access all encoder hidden states.

Seq2Seq with Attention In a sequence-to-sequence model, an encoder RNN processes the input sequence $\mathbf{x}_{1:T}$ and produces hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_T$. A decoder RNN generates the output sequence $\mathbf{y}_{1:T'}$. At each decoder step s, an attention mechanism computes a context vector $\mathbf{c}_s$ as a weighted sum of encoder states:

$$e_{s,t} = \text{score}(\mathbf{d}_{s-1}, \mathbf{h}_t), \quad \alpha_{s,t} = \frac{\exp(e_{s,t})}{\sum_{t'=1}^{T} \exp(e_{s,t'})}, \quad \mathbf{c}_s = \sum_{t=1}^{T} \alpha_{s,t} \mathbf{h}_t.$$

The score function is often additive (Bahdanau) or multiplicative (Luong). The context vector is then combined with the decoder's hidden state to predict the next output.

Definition 0.13 (Additive Attention)**.**

$$e_{s,t} = \mathbf{v}^\top \tanh(\mathbf{W}_d \mathbf{d}_{s-1} + \mathbf{W}_h \mathbf{h}_t + \mathbf{b}),$$

where $\mathbf{v}, \mathbf{W}_d, \mathbf{W}_h, \mathbf{b}$ *are learnable parameters.*

Definition 0.14 (Multiplicative (Dot-Product) Attention)**.**

$$e_{s,t} = \mathbf{d}_{s-1}^\top \mathbf{W} \mathbf{h}_t,$$

where $\mathbf{W}$ *is a learned matrix. If* $\mathbf{W}$ *is identity and norms are ignored, it becomes dot-product attention.*

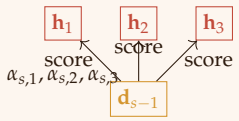


Figure 13: Attention aligns decoder state $\mathbf{d}_{s-1}$ with encoder states $\mathbf{h}_t$ to form a context vector.

Attention has become the cornerstone of modern sequence models. In particular, the Transformer architecture replaces recurrence entirely with self-attention, achieving state-of-the-art results in machine translation, language modeling, and beyond. However, attention is also used inside RNNs to handle long sequences effectively.

Theorem 0.6 (Attention Bypasses Vanishing Gradients)**.** *The attention mechanism provides a direct gradient path from the decoder loss to each encoder hidden state, independent of the distance between them. This effectively solves the vanishing gradient problem for long-range dependencies.*

Regularization for Recurrent Networks

Deep RNNs overfit easily, especially with small datasets. Standard dropout (applied to inputs or outputs) does not work well for recurrent connections because it corrupts the temporal signal. Specialized techniques have been developed.

Variational Dropout Proposed by Gal & Ghahramani (2016), variational dropout applies the same dropout mask across all time steps for each recurrent connection. For a hidden state $\mathbf{h}_t$, the forward pass becomes:

$$\mathbf{h}_t = \phi_h(\mathbf{W}_{hh}(\mathbf{m} \odot \mathbf{h}_{t-1}) + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h),$$

where $\mathbf{m} \sim \text{Bernoulli}(p)$ is a fixed binary mask (same for all t). This is equivalent to a Bayesian interpretation of dropout as approximate variational inference in a deep Gaussian process.

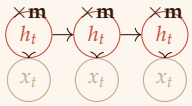


Figure 14: Variational dropout: same mask $\mathbf{m}$ applied at every time step.

Zoneout Zoneout (Krueger et al., 2017) stochastically replaces the computed hidden state with the previous hidden state:

$$\mathbf{h}_t = \mathbf{z}_t \odot \phi_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h) + (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1},$$

where $\mathbf{z}_t \sim \text{Bernoulli}(p)$. This preserves information flow and helps with vanishing gradients.

Layer Normalization for RNNs Layer normalization (Ba et al., 2016) normalises the pre-activation across the feature dimension:

$$\mu_t = \frac{1}{d_h}\sum_{i=1}^{d_h} a_{t,i}, \quad \sigma_t^2 = \frac{1}{d_h}\sum_{i=1}^{d_h}(a_{t,i}-\mu_t)^2, \quad \tilde{\mathbf{a}}_t = \frac{\mathbf{a}_t - \mu_t}{\sqrt{\sigma_t^2+\epsilon}}, \quad \mathbf{h}_t = \phi_h(\gamma\tilde{\mathbf{a}}_t + \beta).$$

It stabilises training and is especially effective with LSTMs.

Remark 0.1. In practice, a combination of variational dropout on recurrent connections and layer normalization is often used in state-of-the-art RNNs (e.g., AWD-LSTM).

Probabilistic Recurrent Neural Networks

Deterministic RNNs map a deterministic input sequence to a deterministic hidden state. For uncertainty quantification and generative modeling, we introduce **stochastic latent variables**.

Variational RNN (VRNN) The VRNN (Chung et al., 2015) augments the RNN hidden state with a latent variable $\mathbf{z}_t$ at each time step. The generative model is:

$$\mathbf{h}_{t-1} = f(\mathbf{x}_{t-1}, \mathbf{z}_{t-1}, \mathbf{h}_{t-2}), \tag{20}$$

$$\mathbf{z}_t \sim \mathcal{N}(\boldsymbol{\mu}_{z,t}, \mathrm{diag}(\boldsymbol{\sigma}_{z,t}^2)), \quad \text{where } [\boldsymbol{\mu}_{z,t}, \log\boldsymbol{\sigma}_{z,t}^2] = \psi(\mathbf{h}_{t-1}), \tag{21}$$

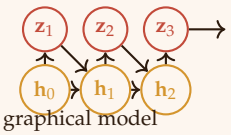
$$\mathbf{x}_t \sim \mathcal{N}(\boldsymbol{\mu}_{x,t}, \mathrm{diag}(\boldsymbol{\sigma}_{x,t}^2)), \quad \text{where } [\boldsymbol{\mu}_{x,t}, \log\boldsymbol{\sigma}_{x,t}^2] = \theta(\mathbf{h}_{t-1}, \mathbf{z}_t). \tag{22}$$

The inference (posterior) network approximates $q(\mathbf{z}_t \mid \mathbf{x}_{\le t}, \mathbf{h}_{t-1})$ with parameters from a neural network ϕ.

The training objective is the sequential evidence lower bound (ELBO):

$$\mathcal{L} = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_t|\cdot)}\big[\log p(\mathbf{x}_t \mid \mathbf{z}_t, \mathbf{h}_{t-1})\big] - \mathrm{KL}\big(q(\mathbf{z}_t \mid \mathbf{x}_{\le t}, \mathbf{h}_{t-1}) \| p(\mathbf{z}_t \mid \mathbf{h}_{t-1})\big).$$

VRNNs can generate highly structured sequences (e.g., handwriting, speech) and quantify predictive uncertainty.



VRNN graphical model

Figure 15: VRNN: deterministic RNN hidden states $\mathbf{h}_t$ and stochastic latents $\mathbf{z}_t$ interact.

Memory Augmented RNNs

Standard RNNs compress all history into a fixed-dimensional hidden state, which limits long-term memory. Memory augmented networks introduce an external memory matrix that the RNN can read from and write to.

Definition 0.15 (Neural Turing Machine (NTM))**.** *An NTM consists of:*

- *A controller (RNN) that outputs read/write vectors.*

- *A memory matrix* $\mathbf{M}_t \in \mathbb{R}^{N \times M}$ *(size* N *vectors of dimension* M*).*
- *Read heads produce content-based and location-based addressing.*

At each step, the controller reads $\mathbf{r}_t = \sum_i \alpha_i^{read} \mathbf{M}_t[i,:]$*, where* $\boldsymbol{\alpha}^{read}$ *is an attention distribution. Writing combines an erase vector* $\mathbf{e}_t$ *and an add vector* $\mathbf{a}_t$*:*

$$\mathbf{M}_{t+1}[i,:] = \mathbf{M}_t[i,:] \odot (1 - \alpha_i^{write} \mathbf{e}_t) + \alpha_i^{write} \mathbf{a}_t.$$

The **Differentiable Neural Computer (DNC)** (Graves et al., 2016) extends the NTM with dynamic memory allocation and temporal linking, enabling it to solve complex algorithmic tasks like graph traversal and reasoning.

Memory augmented RNNs can learn to copy long sequences, sort numbers, and perform reasoning tasks that standard RNNs fail at. However, they are more computationally expensive and harder to train.

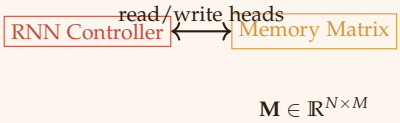


Figure 16: Neural Turing Machine: an RNN controller accesses an external memory matrix.

Graph Recurrent Neural Networks

When data are graphs (e.g., molecules, social networks), sequential ordering is not natural. Graph RNNs generalise the recurrence to propagation over graph edges.

Definition 0.16 (Gated Graph Neural Network (GGNN))**.** *For a graph with nodes* $v \in \mathcal{V}$*, each node has a hidden state* $\mathbf{h}_v^{(t)}$ *that evolves according to:*

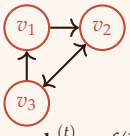
$$\mathbf{a}_v^{(t)} = \sum_{u \in \mathcal{N}(v)} \mathbf{W}_{vu} \mathbf{h}_u^{(t-1)} + \mathbf{W}_x \mathbf{x}_v,$$

where $\mathcal{N}(v)$ *are neighbours of* v*. The state update uses a GRU cell:*

$$\mathbf{h}_v^{(t)} = GRU(\mathbf{h}_v^{(t-1)}, \mathbf{a}_v^{(t)}).$$

After T *steps, the final node states are used for predictions (e.g., graph classification).*

GGNNs are a special case of message-passing neural networks. They can handle cycles and learn long-range dependencies on graphs.



Graph recurrence: $\mathbf{h}_v^{(t)} = f(\mathbf{h}_v^{(t-1)}, \{\mathbf{h}_u^{(t-1)}\}_{u \in \mathcal{N}(v)})$

Figure 17: In a Graph RNN, each node's state is updated using messages from neighbours.

Continuous-Time RNNs: Neural ODEs

Standard RNNs assume a discrete time index. Neural ODEs (Chen et al., 2018) treat the hidden state as a continuous function of time.

Definition 0.17 (ODE-RNN)**.** *The hidden state evolution is governed by an ordinary differential equation:*

$$\frac{d\mathbf{h}(t)}{dt} = f_\theta(\mathbf{h}(t), \mathbf{x}(t)),$$

where $\mathbf{x}(t)$ is a continuous-time input (e.g., interpolated from discrete observations). The state at observation times is obtained by integration:

$$\mathbf{h}(t_n) = \mathbf{h}(t_{n-1}) + \int_{t_{n-1}}^{t_n} f_\theta(\mathbf{h}(\tau), \mathbf{x}(\tau))\, d\tau.$$

This formulation naturally handles irregularly sampled time series. The adjoint sensitivity method allows backpropagation through the ODE solver without storing all intermediate states, reducing memory usage.

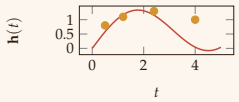


Figure 18: Continuous hidden state $\mathbf{h}(t)$ (line) with discrete observations (dots) at irregular times.

Connection to RNNs: An RNN can be seen as a discretisation of an ODE using a simple Euler step:

$$\mathbf{h}_t - \mathbf{h}_{t-1} = f_\theta(\mathbf{h}_{t-1}, \mathbf{x}_t).$$

Neural ODEs replace the fixed step with an adaptive ODE solver, providing a more flexible and theoretically grounded model for continuous-time dynamics.

Evaluation Metrics for Sequence Models

Measuring performance of sequence models depends on the task. Below are the most common metrics with formal definitions.

Perplexity (Language Models) Given a test sequence $y_{1:T}$ and a model predicting $P(y_t \mid y_{<t})$, perplexity is

$$\text{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P(y_t \mid y_{<t})\right).$$

Lower perplexity indicates better generalisation. For a uniform model over a vocabulary of size V, $\text{PPL} = V$.

Word Error Rate (WER) (Speech Recognition) WER is the edit distance (Levenshtein) between the predicted word sequence $\hat{\mathbf{w}}$ and the reference $\mathbf{w}$, normalised by the reference length:

$$\text{WER} = \frac{\text{Insertions} + \text{Deletions} + \text{Substitutions}}{|\mathbf{w}|}.$$

BLEU Score (Machine Translation) BLEU measures n-gram overlap between candidate and reference translations with a brevity penalty (BP):

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \quad p_n = \frac{\#\text{ of } n\text{-grams in candidate that appear in reference}}{\#\text{ of } n\text{-grams in candidate}}.$$

Typically $N = 4$ and $w_n = 1/N$.

ROUGE (Summarisation) ROUGE (Recall-Oriented Understudy for Gisting Evaluation) computes recall of n-grams and longest common subsequences. ROUGE-N is n-gram recall: $\frac{\sum_{n\text{-gram}\in\text{ref}} \min(\text{count}_{\text{ref}}, \text{count}_{\text{cand}})}{\sum_{n\text{-gram}\in\text{ref}} \text{count}_{\text{ref}}}$.

Remark 0.2. No single metric captures all aspects of quality. BLEU correlates weakly with human judgment; newer metrics like BERTScore (using contextual embeddings) are gaining popularity.

Autoregressive Sequence Generation

Once an RNN is trained, we can generate new sequences by sampling from the learned conditional distribution. This is an **autoregressive** process: each new token is generated based on previously generated tokens.

Greedy Decoding At each step t, choose $\hat{y}_t = \arg\max P(y_t \mid \text{history})$. This is fast but can lead to repetitive or suboptimal sequences.

Beam Search Maintain a set of k (beam width) best candidate sequences. At each step, expand each candidate by all possible next tokens, keep the top k overall. Beam search often produces higher-quality outputs.

Sampling with Temperature The output softmax distribution is often sharpened or flattened using a temperature parameter τ:

$$P(y_t = i \mid \text{history}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)},$$

where z_i are the pre-softmax logits. For $\tau \to 0$, sampling approaches greedy (deterministic); for $\tau = 1$, we recover the original distribution; for $\tau \to \infty$, the distribution becomes uniform. This controls the "creativity" of the generated text.

Example 0.1 (Character-Level RNN Sampling). A character RNN trained on Shakespeare can generate new text by starting with a seed character, then repeatedly sampling the next character from the predicted distribution (with, say, $\tau = 0.8$). The generated text often mimics the style of the original corpus.

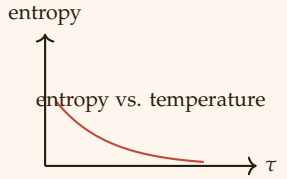


Figure 19: As temperature τ increases, the sampling distribution becomes more uniform (higher entropy).

Open Problems and Limitations of RNNs

Despite their success, RNNs have fundamental limitations that remain active research areas.

Turing Completeness and Precision Siegelmann & Sontag (1995) proved that RNNs with rational weights are Turing-complete. However, this requires infinite precision. In practice, finite precision limits its memory capacity; an RNN with d_h hidden units can only store $\mathcal{O}(d_h \log d_h)$ bits of information, making it impossible to emulate arbitrary long-range dependencies beyond a certain length.

The Expressivity vs. Trainability Tradeoff Deep (stacked) RNNs can represent complex functions, but they suffer from vanishing/exploding gradients even with gating. While LSTMs and GRUs alleviate the issue for sequences of hundreds of steps, they still struggle with dependencies spanning thousands of steps (e.g., language modeling over full books). Attention mechanisms largely solve this but move away from recurrence.

Orthogonal and Unitary RNNs To enforce gradient preservation, researchers have constrained $\mathbf{W}_{hh}$ to be orthogonal or unitary (where eigenvalues lie on the unit circle). Such RNNs guarantee that the Jacobian norm is exactly 1, eliminating vanishing/exploding gradients. However, they are less expressive than full RNNs and training is constrained.

Theorem 0.7 (Unitary RNN Gradient Flow)**.** *If $\mathbf{W}_{hh}$ is unitary ($\mathbf{W}_{hh}^{\mathsf{H}}\mathbf{W}_{hh} = \mathbf{I}$) and ϕ_h is identity, then $\|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}}\| = 1$ for all k.*

The Fundamental Bottleneck of Fixed-Size Hidden States No matter how many units, a fixed-size hidden state cannot store unlimited information. This is the justification for external memory (NTM/DNC) and attention (which accesses all past states). However, attention has quadratic cost in sequence length, leading to the current research frontier: efficient transformers and linear RNNs (e.g., RWKV, S4).

Recurrent vs. Convolutional vs. Transformer Tradeoffs

- RNNs: linear time and memory in T, but weak long-range memory.
- CNNs (e.g., WaveNet): parallel training, but require many layers for long-range dependencies.
- Transformers: excellent long-range memory, but quadratic cost (addressed by sparse attention, linear attention).

No single architecture dominates all regimes; the choice depends on sequence length, hardware, and task structure.

Summary

Recurrent Neural Networks provide a principled framework for modeling sequential data by maintaining an internal state that evolves over time. The training procedure, backpropagation through time, is a direct extension of the backpropagation algorithm for feedforward networks but must handle weight sharing across time. The vanishing and exploding gradient problems are inherent to simple RNNs; gated architectures (LSTM, GRU) are the standard solution. These models form the basis of modern sequence-to-sequence learning, machine translation, speech recognition, and many other applications.

The mathematical tools developed in earlier chapters - probability, MLE, gradient descent, and information theory - all play a role in understanding and training RNNs. The next logical step is to explore attention mechanisms and transformers, which have largely replaced RNNs in many domains but still rely on the same foundational concepts.